Francisco Redondo, ist1106117
Vasco Pereira, ist1103368
Group 16

Lab 4: Reinforcement learning

1. Question 1

The golf problem has a continuous state space where the ball position $x \in [0, 450]$. Applying a standard (tabular) discretized Q-learning approach runs into several fundamental limitations:

Curse of Dimensionality in State Representation:

To use tabular Q-learning, you must discretize x into bins (e.g., 450 bins of 1 meter each, or finer). The Q-table then has size $|\text{bins}| \times N_a$. If you want fine-grained resolution (e.g., 0.1m precision), you need 4500 bins, and the table grows linearly. In higher-dimensional problems this becomes exponential.

Loss of Generalization:

In a discrete table, each state bin is treated independently. If the agent visits position $x = 100.3$m, it learns nothing about $x = 100.7$m, even though they are physically nearly identical and likely require the same action. There is no sharing of information between neighbouring states.

Poor Handling of Unseen States:

The golf ball can land at any real-valued position after each shot. Many exact positions may never be visited during training. A tabular method has no Q-value estimate for unvisited bins, it either returns the initialization or requires impractically long training to cover the space.

Sensitivity to Bin Size Choice:

The discretization resolution introduces a hard trade-off: coarse bins lose precision and lead to poor policies (the agent cannot distinguish “just before the hole” from “far away”); fine bins make the table huge and slow to converge.

2. Question 2

The fitted Q-iteration (FQI) algorithm approximates the Q-function using linear function approximation with fixed features $f(x) = \left[\frac{x}{450}, 1\right]^\top$, such that $Q_\theta(x, a) = \theta^\top f(x)$, where $\theta \in \mathbb{R}^{2\times N_a}$. While this is a significant improvement over discretization, it still has important limitations:

The Q-function is constrained to be linear in the features. This means $Q_\theta(x, a) = \theta_1 \cdot \frac{x}{450} + \theta_2$, which can only represent a straight line over the state space for each action. The true Q-function may be highly non-linear (e.g., the optimal action near the hole $x \approx x_{\text{goal}}$ differs sharply from actions far away), and a linear model cannot capture this complexity.

The feature map $f(x) = \left[\frac{x}{450}, 1\right]^\top$ is fixed and chosen manually. It only encodes the normalized position and a bias term. If the relevant structure of the Q-function requires richer representations (e.g., proximity to the goal or distance relative to wind), these cannot be captured. The model has no way to learn better features automatically.

3. Question 3

Both algorithms log the cumulative cost per episode (number of shots taken). The goal is to minimize this over time — a well-trained agent should consistently reach the hole in fewer shots.

FQI uses an epsilon-greedy policy ($\epsilon = 0.1$) with 2000 iterations and updates the parameters θ every 100 transitions using the least-squares solution $\theta_a^{\text{new}} = (f_a^\top f_a)^{-1} f_a^\top y_a$. Because it uses off-policy data and batch regression, it tends to converge faster in the early stages of training.

REINFORCE uses a softmax policy $\pi_\theta(a|x)$ and updates θ after every full episode using the policy gradient $\theta_{t+1} = \theta_t - \alpha\gamma^t G_t \nabla_\theta \log \pi_{\theta_t}(a_t|x_t)$. Being a Monte Carlo method, it has high variance in its gradient estimates, especially early in training when the policy is nearly uniform, which makes convergence slower and noisier.

In practice, FQI is expected to perform better on this problem for the following reasons:

1. The Q-function parametrization $Q_\theta(x, a) = \theta^\top f(x)$ is the same for both, but FQI directly minimizes the Bellman error via least squares, which is a more stable and sample-efficient update than the noisy Monte Carlo gradient used by REINFORCE.
2. REINFORCE suffers from high variance because G_t is computed from a single trajectory. With only 2000 episodes and a maximum of 30 steps, there is limited opportunity to average out this noise.
3. FQI accumulates all past transitions and re-uses them in each regression update, making better use of the collected data compared to the on-policy episodic updates of REINFORCE.

That said, REINFORCE may eventually match FQI given enough iterations or a lower learning rate, and it has the advantage of naturally handling stochastic policies, which provides built-in exploration without needing an explicit epsilon-greedy mechanism.

4. Question 4

In the original Fitted Q-Iteration (FQI) implementation, we update the weights θ by solving a linear regression problem. The goal was to minimize the error between the predicted Q-values ($f_a\theta$) and the target values (y_a). For this, the choice was to go for the Ordinary Least Squares (OLS) closed-form solution, which requires computing the pseudo-inverse of a matrix:

$$\theta_a = (f_a^\top f_a)^{-1} f_a^\top y_a$$

However, this approach can get a little expensive computationally for very large datasets. An alternative is Mini-Batch Stochastic Gradient Descent (SGD). Instead of calculating the perfect solution in one shot, or calculating the gradient over the entire collected dataset (full-batch GD), Mini-Batch SGD samples a small, random subset of the trajectory data. It iteratively adjusts the weights by taking small steps in the direction of the negative gradient of the Mean Squared Error.

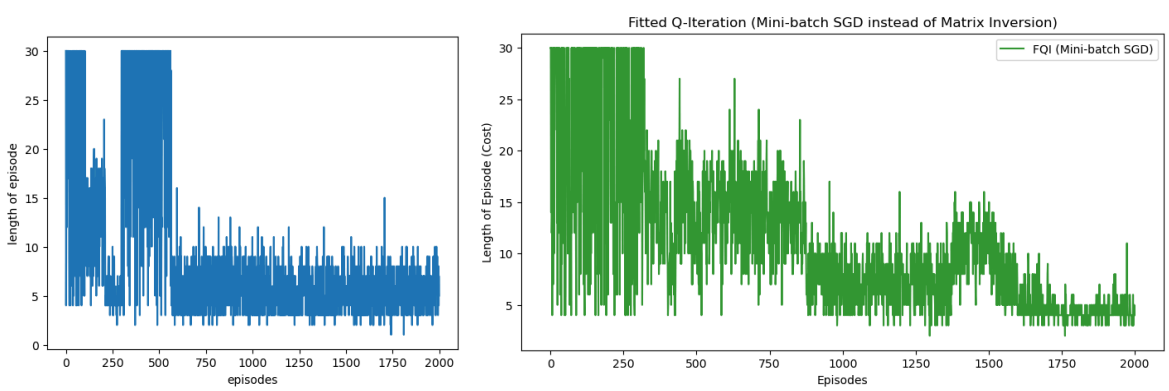

Fig. 1 - FQI trained with 2 different update rules - OLS (left) and Mini-batch SGD (right)

Based on the Fig.1 plots, while it did not necessarily outperform the exact Matrix Inversion, Mini-batch SGD proved to be a highly successful and computationally cheaper approximation.

The Mini-batch SGD plot shows high variance in the early stages (episodes 0–500), where the agent frequently hits the maximum penalty of 30 steps. However, as training progresses, the curve steadily trends downward. By episode 1500, it successfully converges, with the episode length stabilizing in the optimal 3 to 10 step range.

5. Question 5

FQI uses an ϵ-greedy strategy ($\epsilon = 1$). This means 90% of the time, the agent exploits its current linear approximation, and 10% of the time it chooses completely randomly. In contrast, REINFORCE samples actions from a probability distribution and the probabilities are determined by the softmax function.

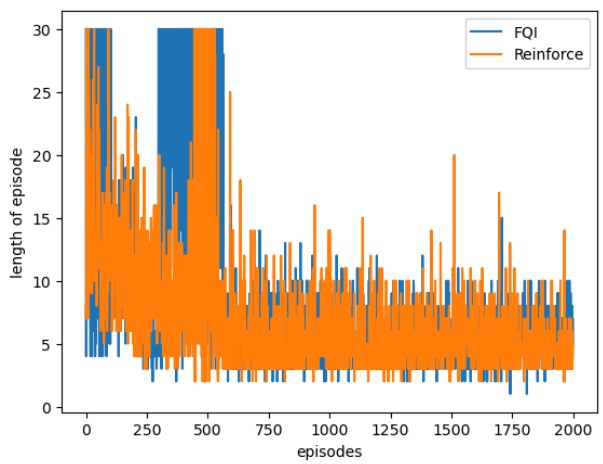

Fig. 2 - FQI vs REINFORCE exploration strategies performance.

We can then conclude that REINFORCE explores more uniformly and naturally, especially early on. At the start of training, the weights (θ) are near zero, meaning the softmax probabilities for all 6 actions are roughly equal. It explores the state space highly uniformly. As it learns, it naturally decays its exploration. If two actions are both good, their probabilities will both remain high, allowing the agent to dynamically explore promising paths. FQI, on the other hand, is rigidly locked into exactly 10% uniform exploration and 90% deterministic exploitation from episode 1 to episode 2000.

6. Question 6

The most classical and effective improvement to REINFORCE is the baseline subtraction. Instead of updating with the raw return G_t, we subtract a baseline $b(x_t)$ from it:

$$\theta_{t+1} = \theta_t - \alpha\gamma^t(G_t - b(x_t))\nabla_\theta \log \pi_{\theta_t}(a_t \mid x_t)$$

The baseline does not introduce bias (it can be shown that $\mathbb{E}[b(x_t)\nabla_\theta \log \pi_\theta(a_t|x_t)] = 0$), but it reduces the variance of the gradient estimate significantly. The most common choice is to use the average return as the baseline.

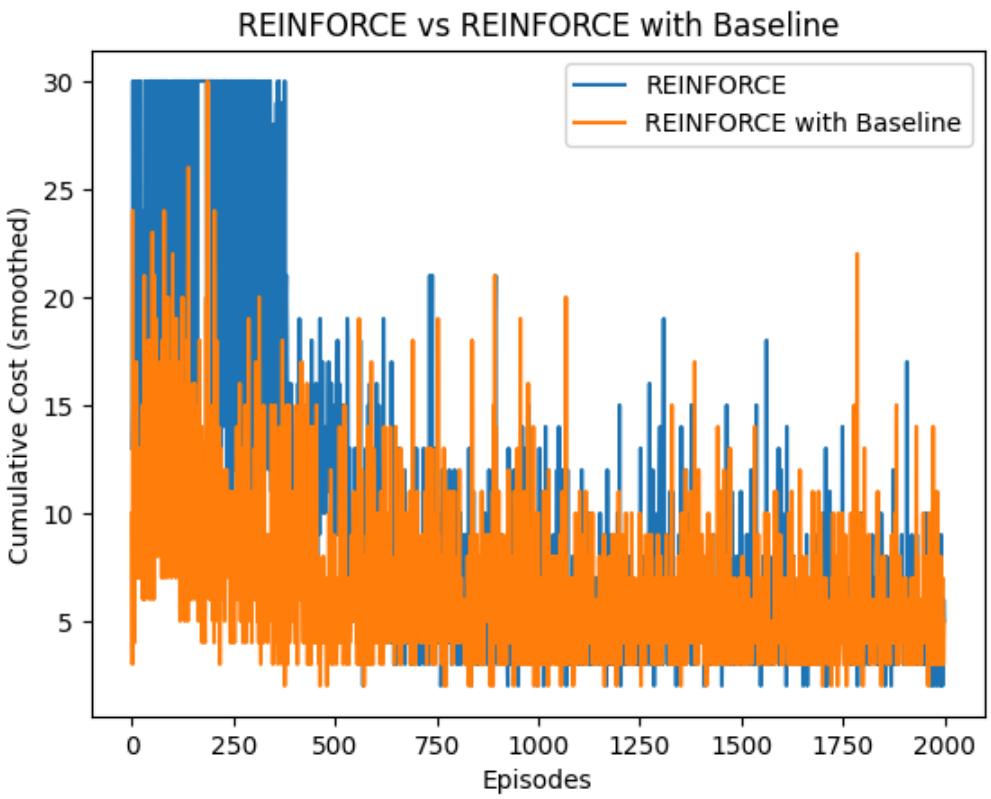

Fig 1. Comparing REINFORCE with REINFORCE with Baseline

Convergence Speed

The REINFORCE with Baseline (orange) drops faster in the early episodes (roughly 0–300), reaching lower cumulative costs sooner than the vanilla REINFORCE (blue). This confirms the theoretical expectation that the baseline reduces variance and allows the policy to improve more quickly in the early stages of training.

Stability

Both methods remain quite noisy throughout training, which is typical of Monte Carlo policy gradient methods. However, the baseline version shows slightly lower and more frequent dips toward the minimum, suggesting it finds good policies more consistently.

Final Performance

By episode 2000, both methods reach similar low cumulative costs (around 2–5 shots), meaning they converge to roughly the same quality policy in the long run. This is consistent with theory, where subtracting a baseline does not change the expected gradient, so both methods converge to the same solution, just at different speeds and with different levels of noise along the way.

The results support the claim that baseline subtraction is a worthwhile improvement to REINFORCE. While the final performance is comparable, the baseline version is more sample efficient and more stable during training. For harder environments or fewer training episodes, this difference would likely be even more pronounced. A natural next step would be to replace the constant baseline with a learned state-dependent value function $V(x)$, which would reduce variance even further by providing a more informative reference point at each state.